

Sistema didattico di analisi e rilevamento del traffico di rete tramite Machine Learning

Introduzione

PurpleMind è un progetto didattico di cybersecurity sviluppato in Python con l'obiettivo di studiare l'analisi del traffico di rete e l'utilizzo del Machine Learning per la rilevazione di eventi anomali o potenzialmente malevoli. Il progetto è stato progettato come un unico sistema chiamato PurpleTeam, all'interno del quale vengono gestite tutte le fasi della pipeline:

generazione o cattura del traffico → creazione del dataset → analisi dei dati → addestramento dei modelli → valutazione → confronto dei risultati.

Il progetto utilizza sia traffico sintetico, generato in un ambiente controllato, sia la possibilità di analizzare traffico reale catturato tramite PyShark. L'obiettivo principale non è realizzare un sistema offensivo, ma creare un laboratorio controllato per comprendere come un sistema di Machine Learning possa essere utilizzato nell'analisi della sicurezza di rete.

Obiettivo del progetto

L'obiettivo principale di PurpleMind è costruire una pipeline completa che permetta di:

- generare eventi di traffico sintetici;
- rappresentare comportamenti normali e sospetti;
- costruire un dataset utilizzabile dal Machine Learning;
- analizzare statisticamente i dati;
- addestrare diversi modelli;
- confrontare le loro prestazioni;
- individuare il modello più efficace secondo determinate metriche;
- sperimentare diverse configurazioni;
- studiare successivamente l'utilizzo di traffico reale.

Il progetto è quindi soprattutto un ambiente di studio.

La domanda principale a cui il sistema cerca di rispondere è:

"Quale modello riesce a distinguere meglio gli eventi benigni dagli eventi classificati come malevoli all'interno dello scenario sperimentale?"

Architettura del progetto

Il progetto è stato diviso in moduli per evitare di concentrare tutto il codice in un unico file.

Ogni modulo ha un compito specifico:

- **config.py** contiene la configurazione.

- **traffic.py** gestisce la generazione del traffico simulato e la cattura autorizzata del traffico reale.
- **dataset.py** gestisce la trasformazione degli eventi in dataset.
- **eda.py** contiene le funzioni per l'analisi esplorativa.
- **models.py** contiene i modelli di Machine Learning.
- **evaluation.py** contiene la valutazione dei modelli.
- **main.py** coordina l'intera pipeline.
- **pteamLab.ipynb** è il laboratorio interattivo utilizzato per studiare le singole fasi.

Perché utilizzare i moduli

Un principiante potrebbe essere portato a scrivere tutto in un unico file.

Per un progetto piccolo può funzionare, ma quando il progetto cresce diventa difficile capire dove modificare una determinata parte. La divisione in moduli permette invece di avere una struttura logica. Per esempio, se voglio modificare il numero di eventi, non devo cercare il numero "500" all'interno di tutto il progetto. Posso modificare semplicemente config.py. Se voglio modificare il modo in cui vengono generati gli eventi, lavoro su traffic.py. Se voglio modificare Random Forest, lavoro su models.py. Questo rende il progetto più semplice da studiare e da estendere.

config.py

Il file config.py contiene i parametri principali della simulazione.

Un esempio è:

```
EVENTI_NORMALI = 500
```

```
EVENTI_ATTACCO = 100
```

In questo modo gli altri moduli possono importare questi valori invece di riscriverli.

Per esempio:

```
from config import EVENTI_NORMALI, EVENTI_ATTACCO
```

Questo principio viene chiamato **configurazione centralizzata**. È particolarmente utile perché permette di effettuare esperimenti rapidamente. Per esempio, possiamo passare da:

500 eventi normali e 100 eventi di attacco

a:

1000 eventi normali e 200 eventi di attacco

modificando solamente la configurazione.

Cosa sono i percorsi dei file

Nel progetto possono essere utilizzate le funzionalità della **libreria standard pathlib**. Un percorso indica dove si trova un file o una cartella.

Per esempio:

data/

indica la cartella in cui possono essere salvati i dataset.

output/

può essere utilizzata per salvare risultati, grafici o report.

L'utilizzo di pathlib permette di gestire questi percorsi in modo più ordinato rispetto alla costruzione manuale delle stringhe.

traffic.py

Il modulo traffic.py si occupa del traffico. Sono previste due modalità differenti. La prima è la generazione di traffico sintetico. La seconda è la possibilità di osservare traffico reale tramite PyShark. Queste due modalità non devono essere confuse. Il traffico sintetico viene creato appositamente per il laboratorio e può essere etichettato perché conosciamo l'origine dell'evento. Il traffico reale, invece, viene semplicemente osservato e non possiede automaticamente una classificazione benign o malicious.

Generazione del traffico sintetico

Per creare gli scenari sintetici viene utilizzato **Scapy**. Scapy è una libreria Python che permette di rappresentare e manipolare pacchetti di rete. Nel progetto viene utilizzata principalmente per costruire scenari controllati. L'obiettivo non è effettuare attacchi reali, ma produrre eventi di rete che possano essere trasformati in dati per il laboratorio di Machine Learning.

Gli eventi possono contenere informazioni come:

- timestamp
- indirizzo IP sorgente
- porta di destinazione
- bytes inviati
- durata
- login falliti

- numero di richieste
- label

Perché utilizzare dati sintetici

Utilizzare inizialmente dati sintetici presenta diversi vantaggi. Prima di tutto conosciamo la **ground truth**. Se creiamo noi un evento classificandolo come malicious, sappiamo che quella è l'etichetta che abbiamo assegnato allo scenario. Questo permette di addestrare e valutare correttamente un classificatore supervisionato. Inoltre possiamo modificare facilmente la difficoltà dell'esperimento.

Possiamo aumentare:

- il numero degli eventi;
- il numero degli eventi malevoli;
- il rumore;
- la sovrapposizione tra le classi;
- la variabilità dei comportamenti.

Il concetto di overlap

Uno dei concetti più importanti del progetto è l'**overlap**. Un dataset troppo semplice potrebbe avere una separazione perfetta.

Per esempio:

- traffico normale → pochi byte
- traffico malevolo → tantissimi byte

In questo caso il modello potrebbe imparare una regola molto semplice. Nella realtà, invece, un comportamento normale può assomigliare parzialmente a un comportamento sospetto. Per questo PurpleMind introduce **sovrapposizione tra le classi**. Ad esempio, alcuni eventi benigni possono avere un **request rate** elevato mentre alcuni eventi classificati come malicious possono avere valori più bassi. Questo rende l'esperimento più interessante. È importante ricordare però che l'overlap simulato non equivale automaticamente alla complessità del traffico reale.

Feature utilizzate

Il dataset utilizza diverse caratteristiche:

- DST_PORT rappresenta la porta di destinazione.
- BYTES_SENT rappresenta la quantità di dati inviati.

- DURATION rappresenta la durata dell'evento o della sessione.
- FAILED_LOGINS rappresenta il numero di tentativi di login falliti.
- REQUEST_RATE rappresenta il numero di richieste generate in un determinato intervallo.
- LABEL rappresenta la classe associata all'evento.
- Nel Machine Learning vengono utilizzate principalmente le feature numeriche.

Dataset

Il modulo **dataset.py** trasforma gli eventi in un **DataFrame** Pandas. Concettualmente:

eventi → DataFrame → dataset

Ogni riga rappresenta un evento. Ogni colonna rappresenta una caratteristica.

Per esempio:

timestamp	src_ip	dst_port	bytes_sent	duration	failed_logins	request_rate	label

Questo formato è particolarmente adatto agli strumenti di analisi dati e Machine Learning utilizzati nel progetto.

Controllo del dataset

Prima di utilizzare il dataset è necessario controllarlo. Vengono utilizzate operazioni come:

- `dataframe.head()` per visualizzare le prime righe.
- `dataframe.shape` per controllare il numero di righe e colonne.
- `dataframe["label"].value_counts()` per controllare la distribuzione delle classi.

Questi controlli sono semplici ma fondamentali. Prima di addestrare un modello dobbiamo sapere con quali dati stiamo lavorando.

EDA Exploratory Data Analysis

Il modulo **eda.py** contiene l'analisi esplorativa. EDA significa Exploratory Data Analysis. Prima di addestrare un modello bisogna cercare di capire i dati.

Vengono analizzati:

- statistiche descrittive;

- distribuzione delle classi;
- distribuzione delle feature;
- boxplot;
- correlazioni;
- relazioni tra variabili.

L'EDA permette di capire se il problema sembra facilmente separabile oppure se esiste una sovrapposizione significativa tra le classi.

Machine Learning

Il modulo models.py contiene i modelli utilizzati. La prima versione del progetto utilizza tre approcci:

- **Logistic Regression**
- **Random Forest**
- **Isolation Forest**

La scelta permette di confrontare modelli differenti. Non si parte direttamente dal modello più complesso. Prima viene utilizzato un modello baseline e successivamente vengono introdotti modelli più avanzati.

Logistic Regression

Logistic Regression viene utilizzata come baseline. È un modello relativamente semplice e permette di avere un punto di riferimento. Il modello viene addestrato utilizzando: **X_train e y_train**. Successivamente viene utilizzato X_test per effettuare le predizioni. Il risultato è una classificazione degli eventi.

Random Forest

Random Forest rappresenta il secondo modello. È composto da numerosi alberi decisionali. Il modello cerca di combinare le decisioni dei diversi alberi. Nel progetto viene utilizzato per verificare se un approccio più complesso riesce a ottenere risultati migliori rispetto alla baseline. Un risultato migliore non deve essere dato per scontato. Deve essere verificato attraverso gli esperimenti.

Isolation Forest

Isolation Forest viene utilizzato per studiare l'Anomaly Detection. Questo modello è concettualmente diverso dai classificatori supervisionati. L'obiettivo è identificare osservazioni che risultano anomale rispetto al comportamento generale dei dati. È quindi interessante confrontare: modelli supervisionati contro modello di anomaly detection. Questo confronto

permette di comprendere le differenze tra classificazione supervisionata e rilevamento delle anomalie.

Preparazione dei dati

Prima dell'addestramento la label viene convertita in valori numerici. Per esempio:

- benign = 0
- malicious = 1

Successivamente vengono separate:

- X = feature
- y = target

Il dataset viene poi diviso in training set e test set.

Training e Test

La configurazione iniziale utilizza una divisione:

- 80% training
- 20% test

Il training set viene utilizzato per l'apprendimento. Il test set viene utilizzato per verificare la capacità di generalizzazione. Questo permette di evitare di valutare il modello solamente sugli stessi dati utilizzati durante l'addestramento.

Valutazione dei modelli

Il modulo **evaluation.py** contiene la parte di valutazione. Vengono utilizzate:

- Accuracy
- Precision
- Recall
- Confusion Matrix
- False Positives
- False Negatives
- Classification Report

Queste metriche permettono di confrontare i modelli.

Accuracy

Accuracy misura quante predizioni sono complessivamente corrette. È una metrica utile, ma non deve essere utilizzata da sola. In particolare, con dataset sbilanciati può dare un'impressione eccessivamente positiva delle prestazioni.

Precision

Precision misura quanto sono affidabili le predizioni positive. In termini semplici:

quando il modello segnala un evento come malicious, quanto spesso ha ragione? Una precision bassa può significare molti falsi allarmi.

Recall

Recall misura quanti degli eventi realmente malicious vengono rilevati. È particolarmente importante nel contesto della sicurezza. Se un attacco viene classificato come benign, abbiamo un **False Negative**. Per questo, nella prima versione di PurpleMind, il Recall viene utilizzato come criterio principale per scegliere il modello migliore.

Confusion Matrix

La confusion matrix permette di osservare più chiaramente gli errori. La confusion matrix permette quindi di capire non solo quante predizioni sono corrette, ma anche quale tipo di errore il modello sta commettendo.

Purple Evaluation Engine

La parte PurpleTeam confronta i risultati dei tre modelli. Il risultato può essere rappresentato in una tabella contenente:

- Model
- Accuracy
- Precision
- Recall
- False Positives
- False Negatives

Successivamente il sistema identifica il modello con il Recall più elevato. Questo costituisce la prima versione del Purple Evaluation Engine. In futuro il criterio potrà essere reso più complesso.

main.py

main.py è il punto di ingresso del progetto. Il file non dovrebbe contenere tutta la logica. **Il suo compito è coordinare i moduli.**

La pipeline è:

1. caricamento configurazione;
2. generazione o acquisizione degli eventi;
3. creazione del dataset;
4. EDA;
5. preparazione dei dati;
6. train/test split;
7. Logistic Regression;
8. Random Forest;
9. Isolation Forest;
10. valutazione;
11. confronto;
12. PurpleTeam Verdict.

pteamLab.ipynb

pteamLab.ipynb è il laboratorio interattivo. Il notebook e main.py eseguono sostanzialmente la stessa pipeline, ma hanno scopi diversi. main.py esegue automaticamente il progetto. pteamLab.ipynb permette invece di eseguire una fase alla volta e osservare i risultati.

Per un neofita il notebook è particolarmente utile perché permette di modificare una cella e vedere immediatamente cosa cambia.

Librerie utilizzate

Il progetto utilizza:

- **Pandas** per la gestione dei dati tabellari.
- **NumPy** per operazioni numeriche.
- **Scikit-learn** per Machine Learning e metriche.
- **Matplotlib** per la visualizzazione.
- **Seaborn** per grafici statistici.
- **Scapy** per la generazione e rappresentazione del traffico sintetico.
- **PyShark** per l'analisi del traffico catturato tramite Wireshark/TShark.

Le librerie standard Python vengono utilizzate per funzionalità generali come gestione dei percorsi, casualità e date.

Avvio del progetto

Dalla cartella principale del progetto: **python main.py**. Questo avvia la pipeline completa.

Il notebook può invece essere utilizzato per lavorare in maniera interattiva.

Come sperimentare

Il progetto è stato pensato anche come laboratorio. Non bisogna limitarsi a eseguire il codice. È consigliabile modificare una variabile alla volta e osservare il risultato.

Per esempio:

- EVENTI_NORMALI = 500
- EVENTI_ATTACCO = 100

può diventare:

- EVENTI_NORMALI = 1000
- EVENTI_ATTACCO = 200

Successivamente si confrontano i risultati.

Esperimenti consigliati

- 1) Un primo esperimento consiste nell'aumentare il numero di eventi.
- 2) Un secondo consiste nel modificare il rapporto tra eventi normali e malicious.
- 3) Un terzo consiste nell'aumentare l'overlap.
- 4) Un quarto consiste nel modificare gli iperparametri di Random Forest.
- 5) Un quinto consiste nel confrontare Random Forest e Logistic Regression.
- 6) Un sesto consiste nel confrontare i classificatori supervisionati con Isolation Forest.

Per ogni esperimento è utile annotare:

- configurazione utilizzata;
- numero di eventi;
- risultati;
- Recall;
- Precision;
- False Positive;

- False Negative;
- modello vincitore.

In questo modo il progetto diventa anche un vero quaderno di esperimenti.

Cattura reale con PyShark

La cattura reale deve essere considerata separatamente dalla generazione sintetica. PyShark permette di leggere pacchetti catturati tramite l'infrastruttura di Wireshark/Tshark. Tuttavia, un pacchetto reale non possiede automaticamente una label. Non possiamo quindi affermare automaticamente: "questo pacchetto è malicious". Per utilizzare traffico reale in un classificatore supervisionato è necessario avere una modalità affidabile per assegnare le etichette.

Utilizzo sicuro del traffico reale

La cattura reale deve essere effettuata esclusivamente su reti e sistemi per i quali si dispone di autorizzazione.

Per lo studio iniziale è consigliabile utilizzare:

- traffico generato dalla propria macchina;
- una rete di laboratorio;
- una macchina virtuale;
- un ambiente isolato;
- dataset pubblici autorizzati.

Non bisogna utilizzare il progetto per intercettare traffico di terzi.

La modalità sintetica rimane quindi il punto di partenza consigliato.

Addestramento sicuro

L'addestramento del modello è un'operazione locale sui dati. Il comando: **python main.py** esegue l'addestramento utilizzando il dataset generato dal progetto. Non è necessario eseguire attacchi reali per addestrare i modelli.

Gli scenari sintetici sono sufficienti per studiare:

- classificazione;
- anomaly detection;
- metriche;
- overfitting;
- generalizzazione;
- confronto tra modelli.

Data Leakage

Un concetto fondamentale da conoscere è il Data Leakage. Il Data Leakage avviene quando il modello riceve durante l'addestramento informazioni che non dovrebbe avere. Per esempio, la label non deve essere utilizzata come feature.

La struttura corretta è:

- X = feature
- y = label

Il modello deve imparare dalle feature e cercare di prevedere y .

Overfitting

Un altro concetto importante è l'overfitting. Un modello può imparare molto bene il training set ma funzionare male su dati nuovi. Per questo utilizziamo il test set. Il nostro obiettivo non è ottenere semplicemente un numero elevato sul training. Vogliamo che il modello generalizzi.

Roadmap futura

Il progetto può essere sviluppato progressivamente.

- 1) Prima fase: dataset sintetico, EDA, tre modelli e confronto.
- 2) Seconda fase: configurazione dinamica e aumento della difficoltà.
- 3) Terza fase: nuovi scenari sintetici e maggiore variabilità.
- 4) Quarta fase: retraining e confronto automatico.
- 5) Quinta fase: monitoraggio più vicino a un ambiente SOC.

Possibili sviluppi futuri includono:

- adaptive difficulty;
- concept drift;
- ensemble detection;
- dashboard;
- monitoraggio temporale;
- mapping MITRE ATT&CK;
- graph-based network simulation;
- event streaming;
- reinforcement learning.

Queste funzionalità devono essere introdotte gradualmente.

Conclusione

PurpleMind nasce come laboratorio didattico per comprendere il collegamento tra cybersecurity, analisi dei dati e Machine Learning. Il progetto parte da una simulazione controllata perché permette di conoscere la ground truth e comprendere facilmente il funzionamento della pipeline. Successivamente introduce la possibilità di osservare traffico reale, mantenendo però separato il problema della cattura dal problema dell'etichettatura. La parte centrale del progetto non è solamente scegliere un algoritmo.

L'obiettivo è comprendere l'intero processo:

dati → analisi → modello → predizione → valutazione → esperimento → miglioramento.

Questa struttura costituisce la base di PurpleMind e permette di estendere il progetto gradualmente senza perdere la comprensione del funzionamento dei singoli componenti. Il valore del progetto sta proprio nel collegamento tra queste competenze. Durante lo sviluppo del progetto sono state applicate diverse competenze di programmazione Python. Una delle competenze principali è stata la suddivisione del programma in moduli. Il progetto non contiene tutta la logica in un unico file, ma utilizza diversi moduli con responsabilità specifiche:

- **config.py** per la configurazione;
- **traffic.py** per il traffico;
- **dataset.py** per la gestione dei dati;
- **eda.py** per l'analisi;
- **models.py** per i modelli;
- **evaluation.py** per la valutazione;
- **main.py** per coordinare la pipeline.

Questo permette di comprendere il concetto di modularità. Ho imparato quindi che un programma non deve essere necessariamente scritto tutto nello stesso file, ma può essere diviso in componenti più piccoli e comprensibili.

Utilizzo delle librerie Python

Il progetto utilizza diverse librerie e questo ha permesso di capire come integrare strumenti diversi all'interno dello stesso programma.

- **Pandas** viene utilizzato per lavorare con i dataset.

- **NumPy** viene utilizzato per le operazioni numeriche.
- **Matplotlib e Seaborn** vengono utilizzati per la visualizzazione.
- **Scikit-learn** viene utilizzato per Machine Learning e metriche.
- **Scapy** viene utilizzato per lavorare con pacchetti e scenari di rete simulati.
- **PyShark** viene utilizzato per analizzare traffico di rete catturato.

Questa parte del progetto insegna una competenza importante nella programmazione reale: non è necessario implementare tutto da zero, ma bisogna saper scegliere e utilizzare correttamente le librerie disponibili.

Competenze di networking

Il progetto collega la programmazione Python al funzionamento delle reti. Attraverso gli eventi di rete vengono studiate informazioni come:

- indirizzo IP sorgente;
- porta di destinazione;
- quantità di dati trasferiti;
- durata della comunicazione;
- frequenza delle richieste.

Questi dati permettono di trasformare un concetto di rete in dati analizzabili da un programma. Questo rappresenta un collegamento importante tra networking e programmazione. Un pacchetto o un evento di rete può essere osservato, trasformato in informazioni strutturate e successivamente analizzato con Python.

Scapy e pacchetti di rete

L'utilizzo di Scapy permette di comprendere meglio la struttura dei pacchetti di rete. Non si tratta più solamente di utilizzare la rete come utente, ma di osservare alcuni dei dati che compongono le comunicazioni. Questo permette di comprendere meglio concetti come:

- IP;
- TCP;
- UDP;
- porte;
- indirizzi sorgente e destinazione;
- traffico di rete.

Nel progetto Scapy viene utilizzato principalmente all'interno di scenari controllati per produrre traffico utile al laboratorio.

PyShark e traffico reale

PyShark introduce un secondo livello di apprendimento. Con i dati sintetici conosco già la situazione che sto creando. Con una cattura reale, invece, osservo ciò che sta effettivamente accadendo sulla rete. Questo permette di capire la differenza tra: "creare un evento di rete" e "osservare un evento di rete". La cattura reale introduce inoltre problemi più realistici, come rumore, variabilità e necessità di interpretare correttamente i dati.

Data Science

Il progetto permette anche di sviluppare competenze di Data Science. Il traffico viene trasformato in un dataset tabellare. Successivamente vengono analizzati:

- numero di eventi;
- distribuzione delle classi;
- valori delle feature;
- correlazioni;
- distribuzioni;
- possibili differenze tra classi.

Ho quindi imparato che prima di utilizzare un modello di Machine Learning è necessario comprendere i dati.

Machine Learning

Il progetto introduce tre modelli differenti. **Logistic Regression** viene utilizzata come modello baseline. **Random Forest** viene utilizzata come modello più complesso per il confronto.

Isolation Forest viene utilizzata per studiare l'Anomaly Detection. Questo permette di comprendere che Machine Learning non significa semplicemente "addestrare un modello".

È necessario anche:

- preparare i dati;
- dividere training e test;
- scegliere le feature;
- addestrare il modello;
- effettuare predizioni;
- valutare i risultati;
- confrontare modelli differenti.

Valutazione dei modelli

Un'altra competenza importante è la valutazione. Ho imparato a utilizzare:

- Accuracy;
- Precision;
- Recall;
- Confusion Matrix;
- False Positive;
- False Negative.

In particolare, il Recall è importante nel contesto della sicurezza perché permette di capire quanti eventi positivi vengono effettivamente individuati. Questo introduce un concetto fondamentale: un modello non deve essere valutato solamente in base alla percentuale generale di predizioni corrette. Bisogna capire anche quali errori commette.

Competenze trasversali

Oltre alle competenze tecniche, il progetto sviluppa anche capacità di problem solving e pensiero analitico.

È necessario capire:

- come dividere un problema complesso;
- come organizzare il codice;
- come interpretare gli errori;
- come leggere i risultati;
- come progettare esperimenti;
- come confrontare diverse soluzioni;
- come documentare il lavoro.

Una competenza particolarmente importante è imparare a non considerare il risultato di un modello come una verità assoluta. Un risultato deve essere interpretato in relazione ai dati e all'esperimento utilizzato.